Politecnico di Torino

Algorithms and Programming

Iniziato	martedì, 21 dicembre 2021, 17:17
Stato	Completato
Terminato	martedì, 21 dicembre 2021, 17:17
Tempo impiegato	7 secondi
Valutazione	**0,00** su un massimo di 16,00 (**0**%)

Domanda 1

Risposta non data

Punteggio max.: 4,00

In a linked list each node points to both the right-hand side and the left-hand side nodes. Each node stores a single string of undefined length and the two pointers. The list is accessible starting from the head pointer on the left extreme, as well as starting from the tail pointer on the right extreme. Strings are stored alphabetically ordered from left to right.
Write the function **insert** to insert a new string str in the list at the correct (ordered) position.
The node data structure and the function prototypes are the following:

```
typedef struct element_s {
  char *name;
  struct element_s *left;
  struct element_s *right;
} element_t;
```

```
void insert (element_t **head, element_t **tail, char *str);
```

Take particular care to: 1) manipulate empty lists, 2) manipulate the left and the right pointers of all elements involved in the insertion, and 3) dynamically allocated all required ADTs. Please, report the entire C code and explain (in plain English language) the logic of the code.

Domanda 2

Risposta non data

Punteggio max.: 5,00

A file specifies an electric board on which a certain number of devices (e.g., resistors, diodes, etc.) are placed along a **straight line**. Among these components, the string "gnd" represents the electric ground. As the designer wants each component as close as possible to a ground sink, he/she must write a C function to evaluate this distance in the following way.

The first line of the file specifies the number of components plus the number of ground points. Each component is reported on a different line, and it is followed by a number reporting its position along the straight line, i.e., the distance (number of millimeters) from it and the origin of the line. The distance between any two components is given by the difference of their distances with respect to the origin of the straight line. The following is an example of such a file:

```
4 2
gnd 0
resistor 35
diode 60
transistor 78
gnd 94
inductor 112
```

Write function **minDist** which receives the file name as a parameter and it returns the sum of the distances from **every** component to the **closest** gnd point. In other words, for each component (notice that ground points gnd are not components) the function must compute the distance to the closest ground point. Then, it must sum the minimum distance for all components and return it.

In the previous case, the components have the following distance from the closest gnd point: 35 (resistor), 34 (diode), 16 (transistor), and 18 (inductor). The program must return the sum of these values.

The function prototype is the following:

```
float minDist (char *name);
```

Implement function **minDist** in C code such it runs in **linear time** with respect to the number of components and ground points. In other words, the function **minDist** must run in O(N) time, where N is the number of components plus the number of ground points (gnd). Specify (in plain English language) which is the key idea adopted. Solutions not respecting this complexity requirement (e.g., with quadratic costs) will be penalized.

Suggestion: Use a temporary data structure to store intermediate information.

Domanda 3

Risposta non data

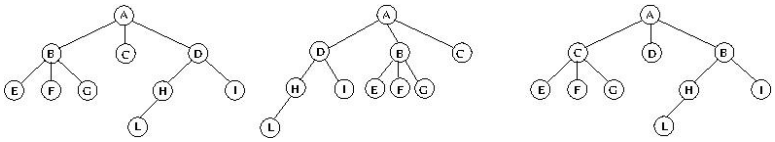
Punteggio max.: 7,00

A n-ary tree is defined using the data structure reported below. Nodes have a string as a key, and a number of children equal to n_child. The array child individuates these children.

```
typedef struct node_s node_t;
struct node_s {
  char *key;
  int n_child;
  node_t **child;
};
```

For each tree, there is a corresponding set of paths, which is the set of all paths starting from the root and reaching all leaves.
Two trees are defined to be equivalent if and only if they generate exactly the same set of paths.

For example, the leftmost tree and the one in the middle are equivalent but the rightmost is not. The two trees on the left-hand side of the picture generate the following set of paths: {ABE, ABF, ABG, AC, ADHL, ADI}. On the contrary, the rightmost tree generates the set of paths: {ACE, ACF, ACG, AD, ABHL, ABI}. Notice that, in this example, all keys are single characters only for the sake of mutual understanding. More generally, keys are strings of unknown length.



Write the function

```
int equivalent (node_t *root1, node_t *root2);
```

to verify whether two trees are equivalent or not. The function must return 1 if the two trees are equivalent (they do generate the same set of paths) and must return 0, otherwise.

Notice that it is not advised to store the entire set of paths generated by a tree to solve the problem. Moreover, it is requested a function whose cost is linear in the size of the trees passed as parameters. Less time-effective or memory-effective solutions will be penalized.
